Avionics and Spacionics · Master's in Aerospace Engineering · DETI · University of Aveiro· 2026/27

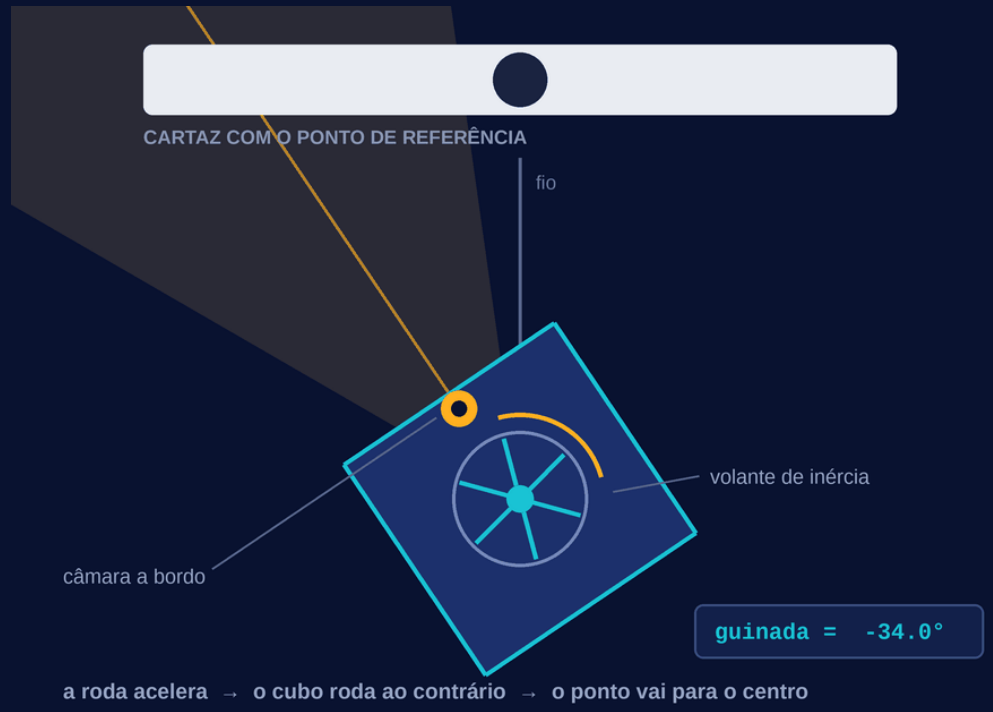
Lecturer: Rodolfo Martins

EMBEDDED SYSTEMS AND CONTROL PROJECT

A cube that learns to point on its own

Attitude control with reaction wheels, guided by vision.

From simulator to hardware, in 7 levels



The cube hangs in a gimbal. It turns because the wheels inside it turn the other way.

CONTEXT

How does a satellite point, with nothing to push against?

A satellite has no ground, no air, no walls. To turn its antenna toward Earth or its telescope toward a star, it uses reaction wheels: it spins a disc inside and the body turns the opposite way. Angular momentum traded, nothing spent.

1. **Hubble Telescope**
   4 reaction wheels point it with fraction-of-an-arcsecond precision.

2. **University CubeSats**
   The same technique, in 10 cm boxes. It is exactly what we are going to build.

3. **Robotics and drones**
   Camera stabilization, balancing robots, arms in free fall.

The whole physics, in one line

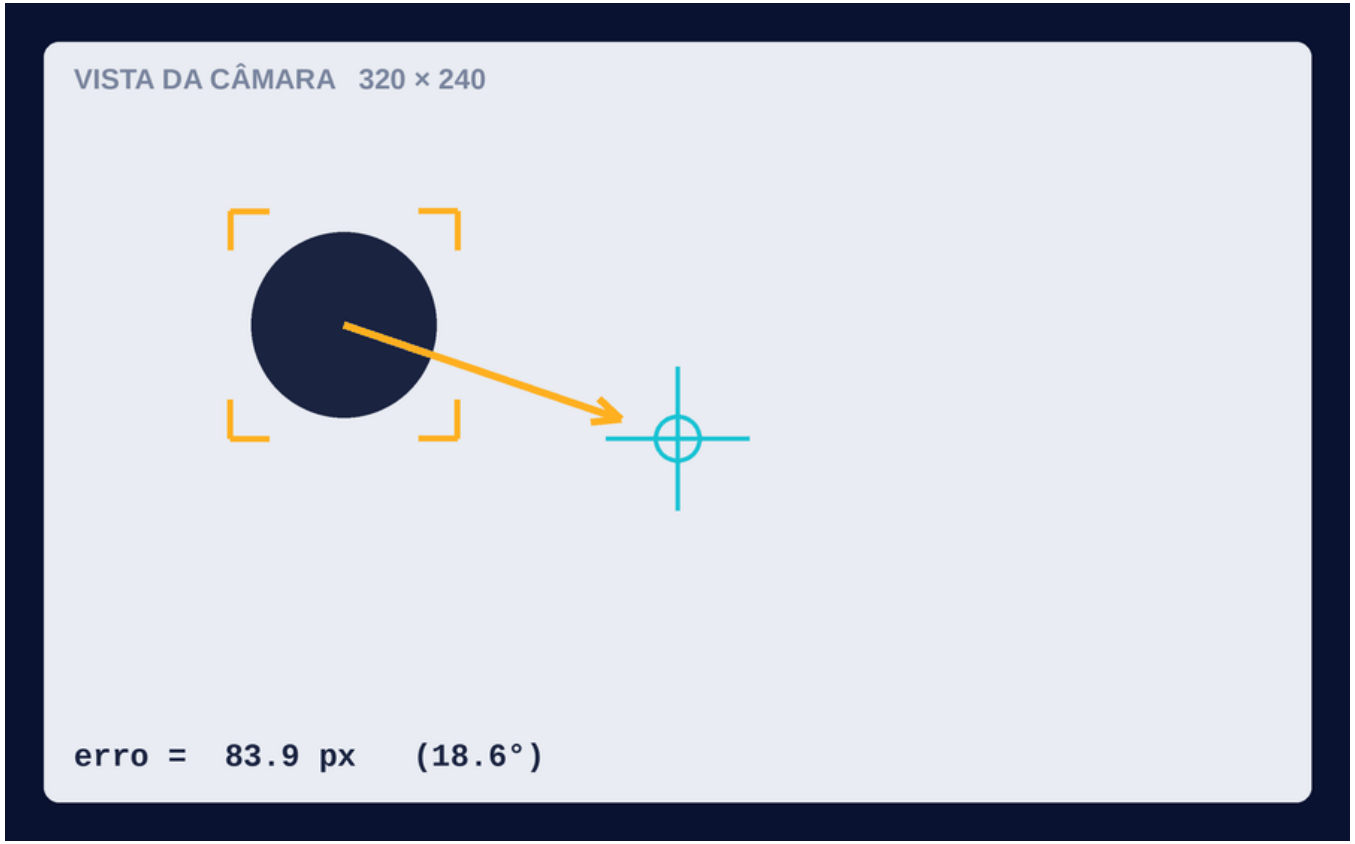
total angular momentum = constant

If the wheel gains rotation one way, the cube gains rotation the other way. There is no push against anything — the system only redistributes what it already has.

No thrusters. No support. Only exchange of rotation.

THE FINAL TARGET

The cube sees the point and turns until it's centred. On its own.



What the cube's camera is seeing, in real time

What counts as success

No one touches the cube

No manual commands. The only input is the camera image.

Under 6 seconds

From 30° off target, it must centre within this time.

Stays there

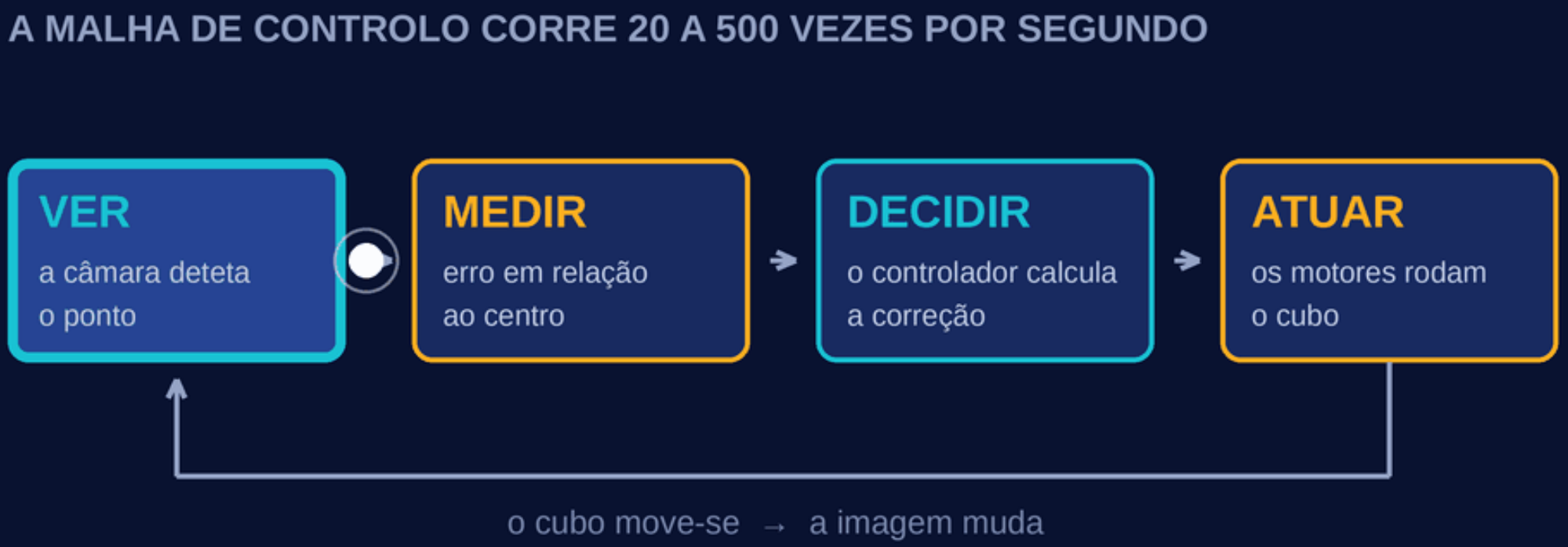
Once centred, it stays within ±2° for a full minute.

Withstands a shove

If it's nudged, it returns to centre without help.

THE PRINCIPLE

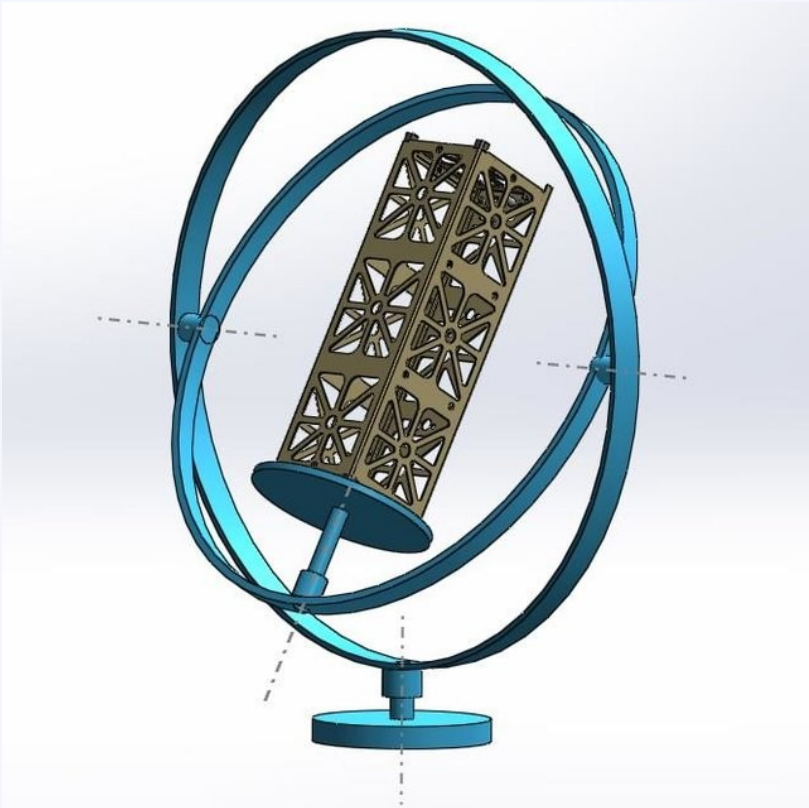
See, measure, decide, act — without stopping.



There is no trick: it is this loop, repeated fast enough to look as if the cube has intent of its own.

THE SUSPENSION

Why the cube is hung like this



Three rings, three freedoms

Each ring frees one rotation. The cube turns in any direction without the structure forcing it — it is the closest thing to an orbit achievable in a classroom.

Balancing is half the project

The centre of mass has to fall on the crossing of the three axes. Off-centre, gravity creates a permanent torque and the wheel wears itself out compensating for it instead of pointing.

Friction is the dominant disturbance

In the bearings lies the biggest difference between this bench and space. It is also what makes level 6 harder than the simulator makes it look.

Without the gimbal there is no demonstration: a fixed cube has nothing to exchange angular momentum with. What is built is a make-believe orbit, where the cube has nothing to push against.

THE AXES

What is controlled (1-axis), and what is only damped

Yaw

CONTROL AXIS

This is where the cube orients itself according to the visual target. It is the axis the reaction wheel commands, and the only one that closes the loop with the camera.

Roll

DAMPING

The suspension creates a natural tendency to return. The goal here is to reduce oscillations, not to point.

Pitch

DAMPING

Pendular dynamics, treated as a disturbance to reject — not as a free attitude axis.

One wheel, one controlled axis.

The software architecture is the same for three axes: adding wheels is adding channels, not rewriting the control.

PERCEPTION

The cube has two senses, and neither is enough alone

Camera

The truth, slowly

It looks for the high-contrast point on the poster and measures how far off centre the target is, in degrees.

It always knows where the centre is, but arrives at ~20 frames per second and with lag.

IMU

The motion, now

Gyroscope and accelerometer say how fast the cube is turning and what orientation it's in.

It responds in milliseconds, but the orientation it integrates keeps accumulating drift.

The camera corrects the IMU's drift. The IMU covers the gaps between frames.

Fusing the two is half the project — and it is what separates a cube that points from a cube that oscillates.

ANATOMY

What goes inside the cube

Camera

Looks at the poster and produces ~20 frames per second.

Microcontroller

The brain. It sees, computes and commands, all inside the cube.

Inertial sensor (IMU)

Says how fast and in what orientation the cube is.

Motor + reaction wheel

The muscle. It spins a disc and the cube turns the other way.

Power electronics

Translates commands into current for the motor.

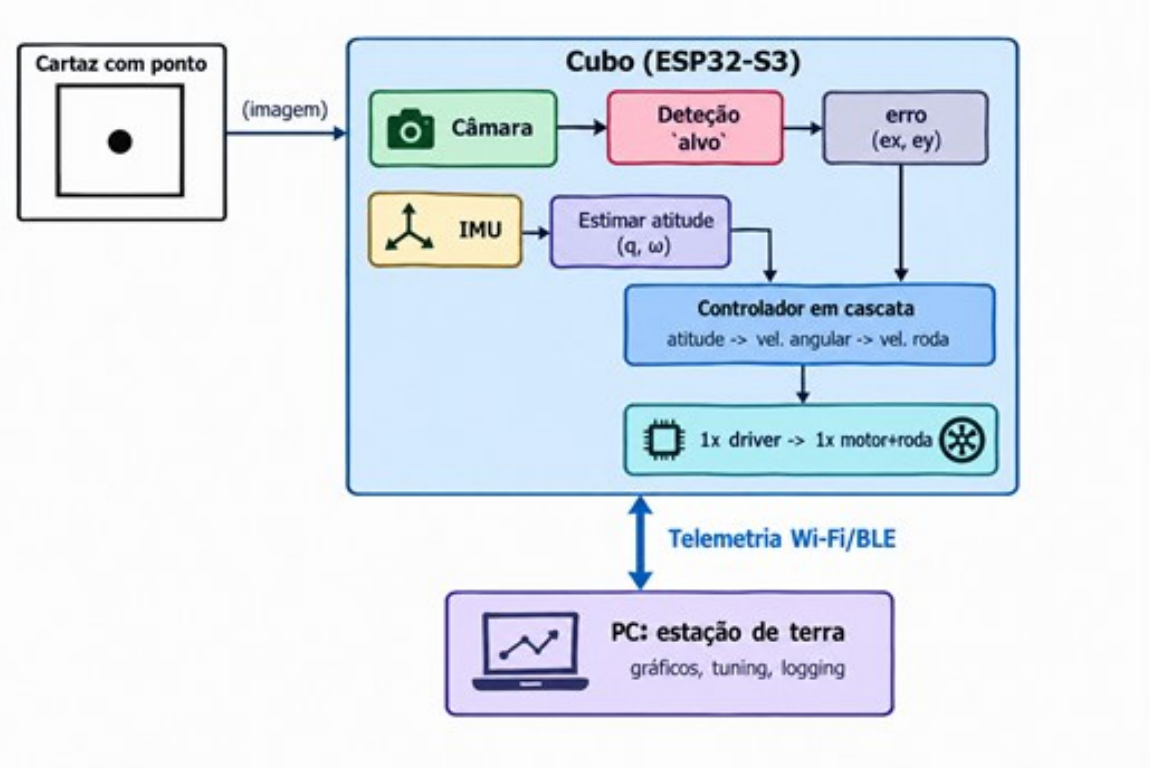
Onboard battery

It has to be self-contained — a cable would twist and stall the cube.

Plus the gimbal, the 3D-printed structure and a poster with a black dot. That's all.

MATERIAL

The shopping list and how it all connects



COMPONENT	REFERENCE
ESP32-S3 Sense XIAO Eval BD + OV3660 camera	SKU 113991115
OV5640 Camera for XIAO ESP32-S3 Sense Eval BD	SKU 114993115
IMU 3-axis gyroscope + 3-axis accelerometer, 5.5 V	SEN10039
3-axis magnetometer ±4/8/12/16 Gs, SPI/I²C	1528-4479-ND
Brushless outrunner motor 5208, Kv330, 600 W	mj5208
moteus r4.11 controller, STM32, 44 V 12 A, CAN-FD	moteus r4.11
Li-Po battery 22.2 V 1400 mAh 6S1P, XT60-F	TAA14006S15X6

General architecture: perception on board, cascade control, telemetry to the ground station.
One motor and one controller — the yaw axis. Structure, gimbal, bearings and target poster are supplied separately.

THE FINAL TEST

Five tests, live, with the cube in front of you

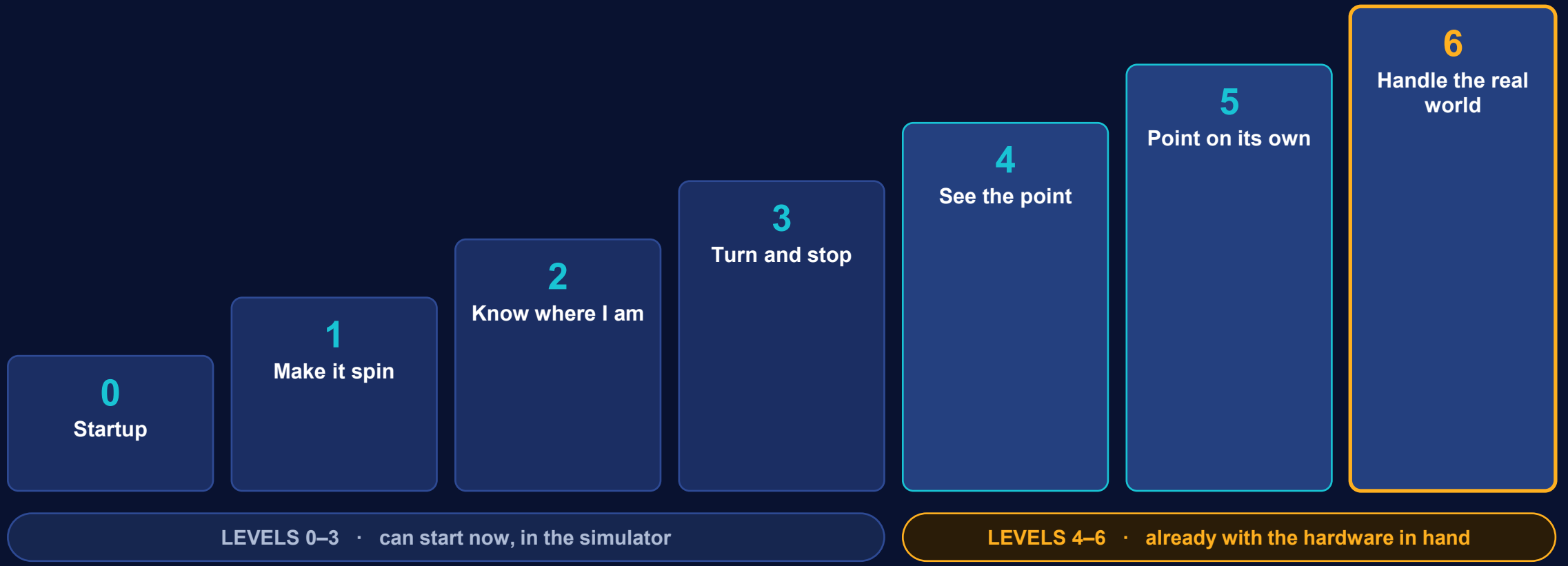
1	**Acquisition**	The cube is released 30° off target.	**Centres in under 6 s.**
2	**Hold**	Left alone, one minute.	**Never leaves ±2° of centre.**
3	**Disturbance**	It's nudged by ~20°.	**Recovers in under 4 s.**
4	**Moving target**	The poster is moved slowly.	**Follows it without losing it.**
5	**Occlusion**	The camera is covered for 2 seconds.	**Doesn't panic. Resumes on its own.**

Each test is measured, not judged. It either passes or it doesn't.

THE PATH

Seven levels. None is skipped.

Each level closes with a 5-minute demo and a number that either meets the criterion or doesn't.



FIRST HALF

Levels 0 to 3 — all of this is done without hardware

0 Startup

Install the environment, build the base project and see telemetry reaching the computer.

PASSES WHEN

The code runs and responds.

1 Make it spin

Command a motor to a requested speed and hold it, despite friction.

PASSES WHEN

Reaches 3000 rpm in <0.5 s, error <2% - TBC

2 Know where I am

Read the inertial sensor and turn noisy signals into a reliable orientation.

PASSES WHEN

Error <2° and drift <2° per minute.

3 Turn and stop

Ask the cube for an angle and off it goes — without overshooting or oscillating.

PASSES WHEN

45° step, settling within ±1°.

Level 3 is already a working attitude control system. It just doesn't see anything yet.

SECOND HALF

Levels 4 to 6 — the cube gains eyes

4

See the point

Find the point in the image and say, in degrees, how far off centre it is.

PASSES WHEN

Hits on >98% of frames, ≥15 per second.

5

Point on its own

Wire the eyes to the muscles: the image error now commands the motors.

PASSES WHEN

From 30° off to centre in <6 s.

6

Handle the real world

Nudges, moving target, covered camera, changing light. The world does not cooperate.

PASSES WHEN

Passes the five final tests.

What makes these three levels hard

The image arrives late

Between the photo and the motor's reaction almost a tenth of a second passes.

The wheels have a limit

A wheel doesn't accelerate forever. When it saturates, the correction ends.

The world is noisy

Shadows, vibration and changing light. It has to keep getting it right.

START NOW

Half the project is done with no hardware at all

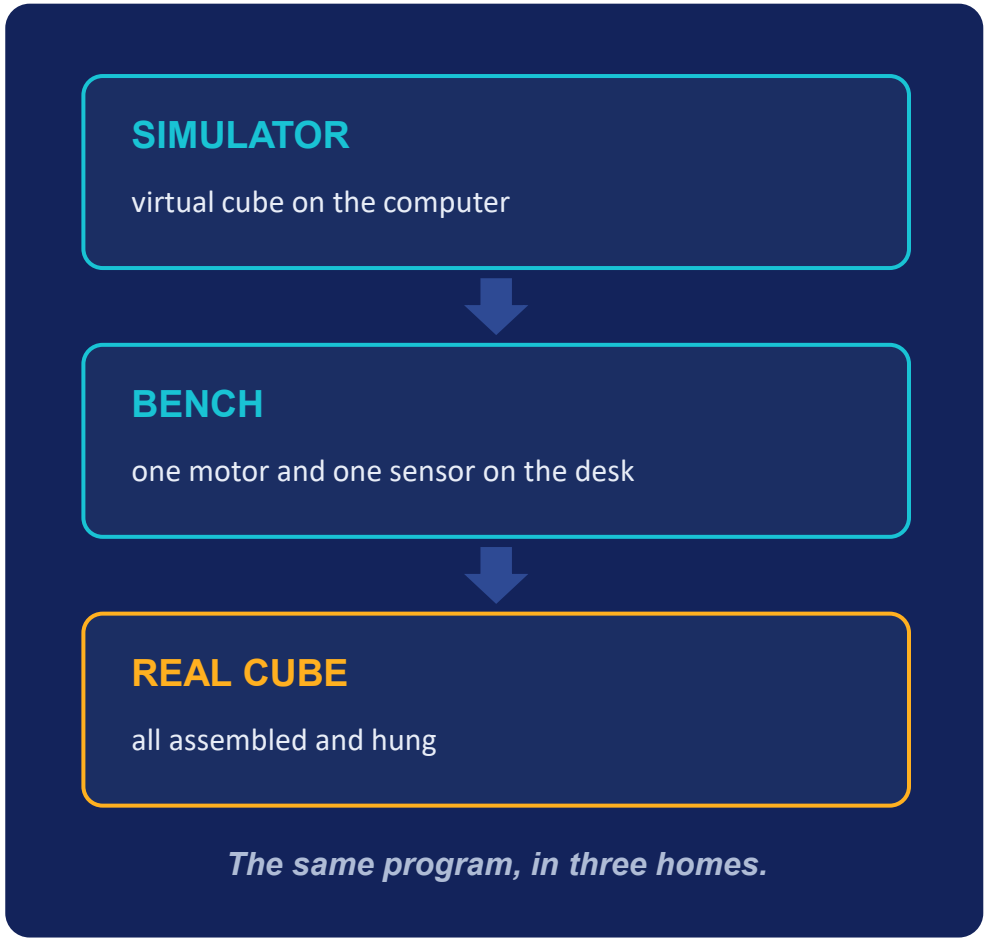
You will receive a simulator of the cube: the physics, the motors, the inertial sensor and even the camera image, all generated on the computer. Your program can't tell the difference — it receives the same data and returns the same commands.

The rule that saves you weeks

The code written in the simulator is the same one that will run on the real cube. Nothing is rewritten — you only swap who supplies the data.

What you can already do in the first weeks

- Tune the controllers until the virtual cube behaves well
- Test the algorithm that finds the point in the image
- See graphs of everything happening, in real time
- Break the system freely — nothing gets damaged



HOW IT IS VALIDATED

Four steps of realism, before the cube exists

MODEL

Physics only, on the computer

How a body that can rotate reacts and what a reaction wheel does to it. Breaking things freely costs nothing.

→

SIL

The real code, in simulation

Software-in-the-loop: the control core that will run on the cube is validated inside the simulator.

→

PIL

The real processor, in simulation

Processor-in-the-loop: the firmware runs on a real ESP32 connected to the simulator. Real-time limits appear.

→

HIL

The real cube, on the bench

Motor, wheel, sensors and structure. Friction, noise, saturation and changing light — the world stops cooperating.

This is another axis, not another scale.

Levels 0 to 6 measure what the cube can already do. These four steps measure how much of the test is already real. They advance in parallel.

WHAT YOU WILL GET

The material handed to you

KIT PER TEAM

- **Cube structure**
  3D-printed parts, fasteners and bearings
- **Board with camera**
  microcontroller and camera on a single board, already aboard
- **Inertial sensor**
  three-axis gyroscope and accelerometer
- **Motor + driver**
  with the reaction wheel and the power electronics
- **Power supply**
  battery, charger and regulators
- **Mount and target**
  suspension structure and poster with the point

You demonstrate, you don't present.

In every class something is shown running.

What isn't in the repository doesn't exist.

Everything integrates into the common project, every week.

How to start - Hardware

List of components to be used in the equipment

- Seeed Studio XIAO ESP32-S3 Sense (https://wiki.seeedstudio.com/xiao_esp32s3_getting_started/) (https://www.seeedstudio.com/XIAO-ESP32S3-Sense-p-5639.html)
- OV5640 Camera for XIAO ESP32-S3 Sense Eval BD (https://www.seeedstudio.com/OV5640-Camera-for-XIAO-ESP32S3-Sense-With-Heat-Sink-p-5739.html)
- IMU 3Axis Gyro, 3 Axis Accel, 5.5V MPU-6050 (https://www.botnroll.com/pt/acelerometros/843-acelarometro-e-giroscopio-3-eixos-.html)
- U SENS 3 Axial Magnetometer ± 4/8/12/16 Gs SPI I2C Breakout (https://mm.digikey.com/Volume0/opasdata/d220001/medias/docus/60/4479_Web.pdf)
- Motor Brushless Controller Moteus r4.11 STM32 44V 12A CAN-FD (https://mjbots.com/products/moteus-r4-11)

Pinout considered for the ESP32-S3

Digital GPIO	Pin Name	Function	Notes
D0	GPIO1	Battery -> ADC1_CH0	Analog input, resistive divider 100k/13k
D1	GPIO2	Buzzer	digital output, active high
D2	GPIO3	MCP2518FD CS	external pull-up (10 k to 3.3 V).
D3	GPIO4	MCP2518FD INT	-
D4	GPIO5	I²C SDA	MPU-6050 + LIS3MDL
D5	GPIO6	I²C SCL	MPU-6050 + LIS3MDL
D6	GPIO43	-	
D7	GPIO44	-	
D8	GPIO7	SPI SCK	
D9	GPIO8	SPI MISO	
D10	GPIO9	SPI MOSI	

How to start - Software

Software list

- **Version control for both the ground station and the firmware**
  - GIT (https://github.com/git-guides/install-git) (Useful for interfacing with GitHub)

- For the simulator (still to be confirmed)
  - MSYS2 (https://www.msys2.org/)
    - In the MSYS2 MINGW64 terminal, install the essential build tools:
      - pacman -S --needed mingw-w64-x86_64-gcc mingw-w64-x86_64-make make

- For Ground Station / Testing / Simulator development
  - Python (https://www.python.org/downloads/windows/)
  - Install libraries for building GUIs, computation and data/graph visualization
    - pip install numpy PyQt5 pyqtgraph

- For firmware development
  - Install VS Code (https://code.visualstudio.com/)
  - Install ESP-IDF (https://docs.espressif.com/projects/esp-idf/en/stable/esp32s3/get-started/windows-setup.html)
  - Install the ESP-IDF extension for VS Code (https://marketplace.visualstudio.com/items?itemName=espressif.esp-idf-extension)
  - Install the driver component for the camera (https://components.espressif.com/components/espressif/esp32-camera/versions/2.1.7/readme)
  - Development guides:
    - Official ESP-IDF website
      - https://docs.espressif.com/projects/esp-idf/en/stable/esp32s3/get-started/index.html
    - Example install/setup tutorials
      - https://www.youtube.com/watch?v=XDDcS7HQNlI
      - https://www.youtube.com/watch?v=FP95rIChNdc
      - https://www.youtube.com/watch?v=h7VfpoJvIpg&list=PL3bNyZYHcRSXm0mNZLiu5hj5ScufC_au9

How to start – Sequence

Plan

- Analyse the component datasheets (see links sent in the component list), possible examples and associated firmware
- Understand how the I2C and CAN-FD protocols work (for interaction with the IMU, magnetometer and motor controller)
- Define the TMTC ICD protocol (communication between Ground Station and cube)
- Describe the firmware
  - Define Device Drivers / Handlers / Applications (block diagram)
  - Communication interface with the ground station,
  - Interface with the motor controller
  - Interface with the IMU
  - Interface with the magnetometer
  - Interface with the camera
  - Description of functional tasks
- Create a mockup of the ground station software and describe the application's features
- Ground station software development
  - Simulate the ground station with simulated cube data (communication level only)
- Firmware development in phases (ideally each of these phases should have a git tag)
    1. IMU and Magnetometer data acquisition and telemetry transmission to the ground station
    2. Motor control, with commands sent by the ground station (e.g. a fixed setpoint for each motor) and controller states sent to the ground station
    3. Autonomous control based on IMU and/or Magnetometer position (position setpoint defined at the ground station) and sending useful information (e.g. errors against setpoint) to the ground station
    4. Camera image acquisition with possible image transmission by telemetry to the ground station
    5. Image processing for target detection
    6. Move the satellite so it finds the point and places it at the centre of the image
    7. Move the satellite so it finds the point and places it off-centre according to a setpoint defined at the ground station

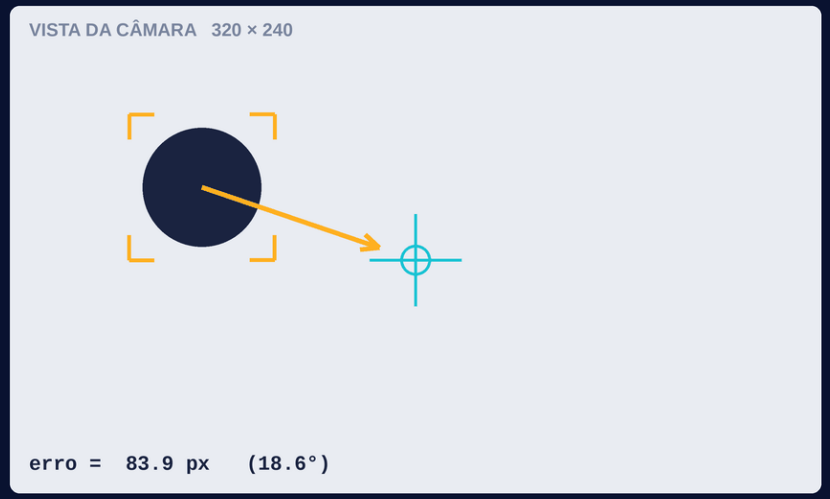
Deliverables (in the report)

- **TMTC ICD**
- **Design Description**
- **Ground Station software**
- **Cube firmware**

AT THE END OF THE SEMESTER

You will hang a cube, take your hands away, and watch it point on its own.

The same technique that points space telescopes, built by hand, in a classroom.



First task

Form teams and install the environment. Level 0 opens this week.